

Question 1 –Avg Velocity

Write a c program to find average velocity when displacement and time are given.

Step1 - Understand the problem

Input

- Displacement d in meters
- Time t in seconds

Output

- Avg velocity v in meters/second

Formula

- $V = d/t$

Step2 - Test case table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result (Pass/Fail)
1	d = 100, t = 20	5.00 m/s	5.00 m/s	Pass
2	d = 50, t = 10	5.00 m/s	5.00 m/s	Pass
3	d = 75, t = 25	3.00 m/s	3.00 m/s	Pass
4	d = 12.5, t = 5	2.50 m/s	2.50 m/s	Pass
5	d = 0, t = 10	0.00 m/s	0.00 m/s	Pass

Step3 - Algorithm

- Start
- Declare displacement,time and velocity
- Read displacement and time
- Calculate velocity=displacement/time
- Print velocity
- Stop

Algorithm evaluation

The algorithm accepts displacement and time as input and calculates average velocity using $\text{velocity} = \text{displacement} / \text{time}$. The calculated results match the expected outputs for the prepared test cases. Hence, the algorithm is correct for the tested inputs.

Step 4- C program

```
#include <stdio.h>
```

```
int main()
{
    double displacement, time, velocity;
    printf("Enter displacement in meters: ");
    scanf("%lf", &displacement);

    printf("Enter time in seconds: ");
    scanf("%lf", &time);
    velocity = displacement / time;
    printf("Average velocity = %.2lf m/s\n", velocity);
    return 0;
}
```

Here, double is used so that decimal inputs can work.

Step 5 – Compilation, execution, and evaluation

Save as: velocity.c

Compile: cc velocity.c -o velocity

Execute: ./velocity

Code Evaluation

The program was compiled and executed successfully. It was tested against the test cases prepared in Step 2. The actual outputs matched the expected outputs; therefore, all prepared test cases passed

Step 6- Screenshot for compilation and execution

```
[puppie@Anvikas-MacBook-Air Desktop % pwd
/Users/puppie/Desktop
[puppie@Anvikas-MacBook-Air Desktop % mkdir HDOC-DAY7
[puppie@Anvikas-MacBook-Air Desktop % HDOC_DAY7
zsh: command not found: HDOC_DAY7
[puppie@Anvikas-MacBook-Air Desktop % pwd
/Users/puppie/Desktop
[puppie@Anvikas-MacBook-Air Desktop % nano velocity.c
[puppie@Anvikas-MacBook-Air Desktop % velocity.c
zsh: command not found: velocity.c
[puppie@Anvikas-MacBook-Air Desktop % cc velocity.c-o velocity
clang: error: no such file or directory: 'velocity.c-o'
clang: error: no such file or directory: 'velocity'
clang: error: no input files
[puppie@Anvikas-MacBook-Air Desktop % cc velocity.c -o velocity
[puppie@Anvikas-MacBook-Air Desktop % ./velocity
Enter displacement in meters: 100
Enter time in seconds: 20
Average velocity = 5.00 m/s
puppie@Anvikas-MacBook-Air Desktop %
```

Question2- power

Write a c program to find power when work done and time are given.

Step 1- Understand the problem

Input

- Work done w in joules
- Time t in seconds

Output-

- Power p in watts

Formula

- $P = w/t$

Step 2- Test case table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result (Pass/Fail)
1	W = 100, t = 10	10.00 W	10.00 W	Pass
2	W = 500, t = 20	25.00 W	25.00 W	Pass
3	W = 75, t = 5	15.00 W	15.00 W	Pass
4	W = 12.5, t = 2.5	5.00 W	5.00 W	Pass
5	W = 0, t = 10	0.00 W	0.00 W	Pass

Step 3 –Algorithm

- Start
- Declare work,time,and power
- Read work done and time
- Calculate power= work/time
- Print power
- Stop

Algorithm evaluation

The algorithm takes work done and time as input and calculates power using power= work/time. The calculated values match the expected outputs for the prepared test cases. Hence,the algorithm is correct for the tested inputs.

Step 4 –C program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    double work, time, power;
```

```
        printf("Enter work done in joules: ");  
        scanf("%lf", &work);
```

```
        printf("Enter time in seconds: ");  
        scanf("%lf", &time);
```

```
power = work / time;  
  
printf("Power = %.2lf watts\n", power);  
  
return 0;  
  
}
```

Step 5 – compilation ,execution and evaluation

Save as : power.c

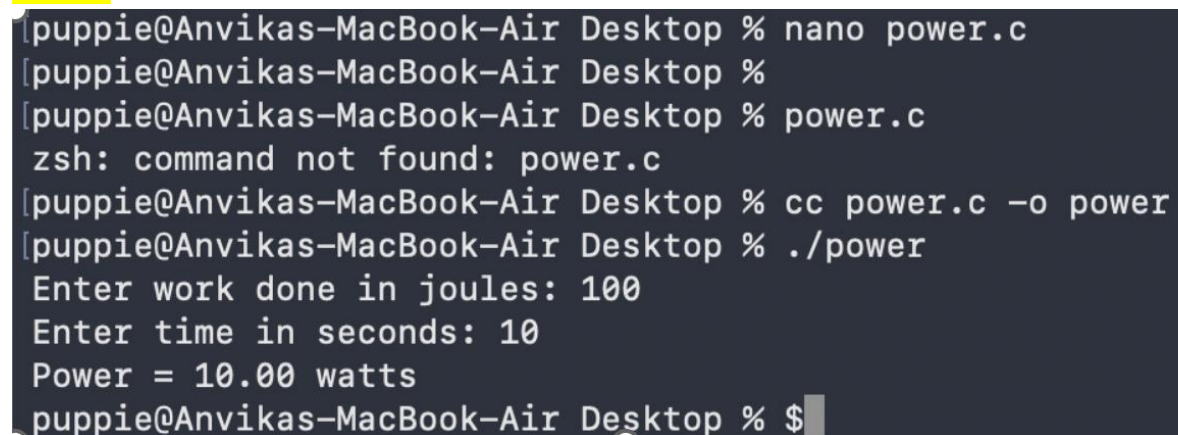
Compile: cc power.c -o power

Run: ./power

Code Evaluation

The program was compiled and executed successfully. It was evaluated against the test cases prepared in Step 2. The actual outputs matched the expected outputs for all prepared test cases; therefore, all test cases passed

Step 6- Screenshot of compilation and execution



```
[puppie@Anvikas-MacBook-Air Desktop % nano power.c  
[puppie@Anvikas-MacBook-Air Desktop %  
[puppie@Anvikas-MacBook-Air Desktop % power.c  
zsh: command not found: power.c  
[puppie@Anvikas-MacBook-Air Desktop % cc power.c -o power  
[puppie@Anvikas-MacBook-Air Desktop % ./power  
Enter work done in joules: 100  
Enter time in seconds: 10  
Power = 10.00 watts  
puppie@Anvikas-MacBook-Air Desktop % $
```

Question 3- Voltage

Write a C program to find voltage when current and resistance are given.

Step 1- Understand the problem

Input

- Current I in amperes
- Resistance r in ohms

Output

- Voltage v in volts

Formula –ohm's law

- $V = I \times R$

Step 2 –Test case table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result (Pass/Fail)
1	I = 2, R = 5	10.00 V	10.00 V	Pass
2	I = 3, R = 10	30.00 V	30.00 V	Pass
3	I = 1.5, R = 4	6.00 V	6.00 V	Pass
4	I = 0.5, R = 20	10.00 V	10.00 V	Pass
5	I = 0, R = 10	0.00 V	0.00 V	Pass

Step 3 – Algorithm

- Start
- Declare current, resistance, and voltage
- Read current and resistance
- Calculate voltage = current x resistance
- Print voltage
- Stop

Algorithm Evaluation

The algorithm takes current and resistance as inputs and calculates voltage using voltage = current × resistance. The calculated values match the expected outputs for the prepared test cases. Hence, the algorithm is correct for the tested inputs.

Step 4- C program

```
#include <stdio.h>

int main()
{
    double current, resistance, voltage;

    printf("Enter current in amperes: ");
    scanf("%lf", &current);

    printf("Enter resistance in ohms: ");
    scanf("%lf", &resistance);

    voltage = current * resistance;

    printf("Voltage = %.2lf volts\n", voltage);

    return 0;
}
```

Step 5 – compilation, execution, and evaluation

Save as: voltage.c

Compile: cc voltage.c -o voltage

Run: ./voltage

Code Evaluation

The program was compiled and executed successfully. It was evaluated against the test cases prepared in Step 2. The actual outputs matched the expected outputs for all prepared test cases; therefore, all test cases passed.

Step 6 – screenshot of compilation and execution

```
[puppie@Anvikas-MacBook-Air Desktop % nano voltage.c
[puppie@Anvikas-MacBook-Air Desktop % voltage.c
zsh: command not found: voltage.c
[puppie@Anvikas-MacBook-Air Desktop % cc voltage.c -o voltage
[puppie@Anvikas-MacBook-Air Desktop % ./voltage
Enter current in amperes: 2
Enter resistance in ohms: 5
Voltage = 10.00 volts
puppie@Anvikas-MacBook-Air Desktop %
```